

COLUMNAR DATABASES — ARCHITECTURE, STORAGE, OPTIMIZATION & USE CASES

v1

Parquet · ORC · Arrow · Iceberg · Delta · Hudi · ClickHouse · Snowflake · BigQuery · DuckDB
file anatomy · encodings · data skipping · vectorized execution · the write path · what actually makes it fast

1 · WHAT COLUMNAR MEANS, AND WHY IT IS FASTER

The definition
A columnar database stores **all values of one column together** on disk, instead of storing each row's fields together. Same data, same schema, same SQL — a different physical arrangement of bytes.

```
// the save three rows, two ways

// Row layout — records contiguous
1, 2026-05-01, US, 120 | 2, 2026-05-01, IN, 200 | 3, ...
one row = data read
one column = touch every record

// Columnar layout — columns contiguous
id | 1, 2, 3 |
date | 2026-05-01, 2026-05-01, 2026-05-01 |
country | US, IN, IN |
amount | 120, 200, 150 |
one column = one sequential read
one row = one read per column
```

That is the whole idea. Everything else on this sheet — encodings, compression, data skipping, vectorized execution, the awkward write path — is a consequence of that one change.

Why it is faster for analytics

An analytical query typically reads **a few columns across very many rows**: `SELECT country, SUM(amount) FROM sales` over a 200-column table. Two things follow.

Property	Consequence
1 · Read less	Only the columns named are touched. On a 200-column table, a three-column query reads roughly three columns' worth of bytes — the other 197 are never fetched.
2 · Compress harder	Adjacent values share a type and often a range, so encodings have structure to exploit. In a row layout an integer sits beside a timestamp beside a string, and there is little to compress.
3 · Skip more	Per-block statistics let whole chunks be eliminated before reading — but only if the data was written in a useful order.
4 · Vectorize	A batch of one column is contiguous memory of one type, which is exactly what SIMD and CPU caches handle well.

The cost is the **minor image**. Reconstructing one whole row means touching every column's storage, and writing one row means rewriting files. Columnar buys analytical throughput by giving up cheap single-row access — that trade is the entire category.

When to choose it

Columnar fits	Row store fits
Scan many rows, few columns	Fetch few rows, many columns
Aggregation and reporting	Point lookup by key
Append-mostly, batch loaded	Frequent single-row insert and update
Wide, denormalised tables	Highly normalised, join-heavy models
Analytics, BI, time series, logs	Transactions, serving APIs

Access pattern decides this, not data size. A ten-million-row table queried with wide scans belongs in a columnar engine; a ten-billion-row table queried only by primary key does not. Ask what the queries look like before asking how big the data is.

The vocabulary

OLTP / OLAP	Transaction processing versus analytical processing — the workload split this whole distinction serves
File format	Parquet, ORC — columnar bytes at rest, engine-independent
Table format	Iceberg, Delta, Hudi — a metadata layer that makes a set of files behave like a table
Engine	ClickHouse, Snowflake, BigQuery, DuckDB — what actually executes the query

The **usual answer** is both. An OLTP database owns the writes and point lookups; changes flow into a columnar store that owns the analytics. Serving both from one engine is the decision most often regretted.

2 · REFERENCE ARCHITECTURE

QUERY PATH — every stage exists to read fewer bytes than the one before it

SQL client → Coordinator → QUERY OPTIMIZER → EXECUTION → STORAGE LAYER → OBJECT STORE / DISK

METADATA / CATALOG LAYER — schema · partition spec · file manifests · column statistics · snapshots and versions · zone maps · bloom filters

EVERY ONE OF THOSE PRUNING STEPS DEPENDS ON HOW THE DATA WAS WRITTEN

PHYSICAL FILE ANATOMY — a Parquet file is four levels deep, and the middle one is the one people forget

THE WRITE PATH — columnar files are immutable, so every update is really a rewrite. This is where columnar systems are actually hard.

THE TRADE THAT DEFINES THE WHOLE CATEGORY

Why it is faster for analytics

An analytical query typically reads **a few columns across very many rows**: `SELECT country, SUM(amount) FROM sales` over a 200-column table. Two things follow.

When to choose it

Columnar fits	Row store fits
Scan many rows, few columns	Fetch few rows, many columns
Aggregation and reporting	Point lookup by key
Append-mostly, batch loaded	Frequent single-row insert and update
Wide, denormalised tables	Highly normalised, join-heavy models
Analytics, BI, time series, logs	Transactions, serving APIs

Access pattern decides this, not data size. A ten-million-row table queried with wide scans belongs in a columnar engine; a ten-billion-row table queried only by primary key does not. Ask what the queries look like before asking how big the data is.

The vocabulary

OLTP / OLAP	Transaction processing versus analytical processing — the workload split this whole distinction serves
File format	Parquet, ORC — columnar bytes at rest, engine-independent
Table format	Iceberg, Delta, Hudi — a metadata layer that makes a set of files behave like a table
Engine	ClickHouse, Snowflake, BigQuery, DuckDB — what actually executes the query

The **usual answer** is both. An OLTP database owns the writes and point lookups; changes flow into a columnar store that owns the analytics. Serving both from one engine is the decision most often regretted.

3 · INSIDE THE FILE

The hierarchy, top down

Level	What it is, and typical size
File	Self-describing and immutable. Opens and closes with the PAR1 magic bytes; all metadata is in the footer.
Row group	A horizontal slice of rows, commonly 128 MB — 1 GB uncompressed. The unit of parallel work and of most skipping. The level the usual diagram omits — without it, "column chunk" has no defined boundary.
Column chunk	All values of one column within one row group. Contiguous on disk, which is what makes a single-column read one sequential range.
Page	The smallest unit, commonly 8 KB — 1 MB . Dictionary pages hold the lookups; data pages hold encoded values. Compression and encoding are applied here.

Read the hierarchy as a filter chain. Parquet eliminates directories, file statistics eliminate files, row-group statistics eliminate slices, the column index eliminates pages, and only then are bytes actually fetched. Each level exists to avoid work at the level below it.

What a reader actually does

```
// reading 2 columns from a 200-column, 1 GB file
1. GET bytes(-81) // footer length + PAR1
2. GET footer // schema, offsets, statistics
3. evaluate predicates against row-group min/max
4. GET only the surviving column chunk, row-group byte ranges — typically a handful of ranged reads
5. decode pages, filter, aggregate

// bytes transferred can be a fraction of one percent
// or the file — but it costs 2 round trips before
// any data moves, which is why file count matters
```

Where statistics live

Location	Enables
Footer	min, max, null count, distinct count — row-group pruning, the coarsest and cheapest.
Column index	Per-page min/max, so a matching row group can still skip most of its pages.
Bloom filter	Equality on high-cardinality columns, where min/max is useless because the range is wide but the value is absent.
Table format manifest	File-level stats, so files are eliminated before a file is read — the only level that avoids a round trip entirely.

Sizing, and the two failure modes

If files are... What goes wrong

Too small The common case

Too large

Target a few hundred megabytes per file with row groups in the low hundreds of megabytes, then let compaction hold it there. Exact values depend on the engine and the storage, but the shape of the trade-off does — and being an order of magnitude out in either direction is far more costly than being imprecise.

Nested and missing data

- Nulls are not stored as values. They are recorded in a definition level, so a mostly-null column costs almost nothing — very different from a row store padding every record.
- Nested structures are flattened into leaf columns, with repetition and definition levels recording the structure. A struct of five fields is five columns, and you can read one without touching the rest.
- Deeply nested arrays cost more on both write and read. If a nested field is filtered on constantly, promoting it to a top-level column is usually the right call.

The footer is a single point of failure. Because metadata is written last, a partially-written file is unreadable rather than partially readable. This is exactly why a table format's atomic commit matters — readers must never see a file that a writer has not finished.

Statistics are only as good as their granularity. One set of min/max per column chunk on a 1 GB row group is a coarse filter. Smaller row groups give finer skipping at the cost of a larger footer — which is the real trade-off behind row-group sizing, not writer memory alone.

Column chunk contiguity is the whole trick. One column's values sit together on disk, so reading it is one sequential range rather than a stride across every row. That single property is what makes projection pushdown a genuine byte-level saving instead of a filter applied after reading everything.

4 · ROW vs COLUMNAR

Row layout — records contiguous

```
1, 2026-05-01, US, 120 | 2, 2026-05-01, IN, 200 | ...
read one row = one seek
read one column = touch every record
```

Columnar layout — columns contiguous

```
id | 1, 2, 3 |
date | 2026-05-01, 2026-05-01, 2026-05-01 |
country | US, IN, IN |
amount | 120, 200, 150 |
read one column = one sequential range
read one row = one seek per column
```

	Row-oriented	Columnar
Reads	Whole rows cheaply	Selected columns cheaply
Writes	Single-row insert and update are cheap	Writes batch into files; single-row update rewrites
Compression	Poor — mixed types adjacent	Strong — one type, similar values adjacent
Point lookup	Index seek, sub-millisecond	Touches every column chunk
Aggregate a column	Reads every row	Reads one column

Compression is a consequence of the layout, not a separate feature. Adjacent values in a column share a type and often a range or a sort order, so encodings have something to exploit. In a row layout an integer sits beside a timestamp beside a string, and there is very little structure to compress.

Ratios are workload-dependent — treat published numbers with care. A low-cardinality, well-sorted column can compress by orders of magnitude; a high-cardinality random one barely at all. The same is true of query speed-ups: the gains come from columns skipped and rows pruned, so it scales with how selective your queries are and how well the data is clustered, not with the product.

5 · DATA SKIPPING AND SORT ORDER

Min/max statistics are useless on unsorted data. They are still written, still correct, and still consulted — and they eliminate nothing, because every row group's range spans the whole domain. Teams conclude "data skipping does not work here" when what actually happened is that nobody controlled write pattern.

```
// INSERTED = 3 row groups, filter: date = '2026-05-01'
R01 min=2026-05-01 max=2026-05-21 = cannot skip
R02 min=2026-05-01 max=2026-05-31 = cannot skip
R03 min=2026-05-01 max=2026-05-31 = cannot skip
reads: 100%
```

```
// SORTED BY date — same data, same query
R01 min=2026-05-01 max=2026-05-18 = read
R02 min=2026-05-11 max=2026-05-28 = skip
R03 min=2026-05-21 max=2026-05-31 = skip
reads: ~33%
```

Re-sorting is expensive, and that is the point of doing it once. Sorting on write, or during a compaction pass, costs CPU once and pays back on every subsequent query. Sorting inside the query pays on every execution and skips nothing, because the files were already read by pattern.

Clustering degrades gracefully; partitioning does not. A sort key that stops matching query patterns still prunes partially. A partition key that stops matching leaves you with millions of directories and a rewrite — which is why locality's ability to evolve the partition spec without rewriting history matters more than it first appears.

Choosing a sort key

- Sort by what you filter on, not by what you group by. Pruning happens on predicates.
- Time first for event data — nearly every analytical query is bounded by a date range, and time is naturally clustered on ingest.
- Then the next most selective filter, typically a tenant, account or region.
- Only the leading column prunes well. A composite sort gives the second column far less benefit, and the third almost none — so choose the order deliberately.
- Multi-dimensional clustering (Z-order, space-filling curves) trades some single-column skipping for usable skipping on several columns. Worth it when queries genuinely filter on different columns; not otherwise.
- High-cardinality equality wants a bloom filter, not a sort key.

Predicates that silently disable pruning. Statistics are compared against literal values, so anything that hides the column from the optimiser costs you the skip:

```
-- little pruning: the column is wrapped in a function
WHERE DATE(event_id) = '2026-05-01'
WHERE CAST(id AS VARCHAR) = '1234'
WHERE UPPER(country) = 'US'

-- pruned: the column is compared directly to literals
WHERE event_id = '2026-05-01'
AND event_id < '2026-05-04'
WHERE id = 1234

-- also weak: leading wildcards, OR across many columns, no joins on the partition column
WHERE name LIKE 'Acme'
```

How to tell whether it is working. Read the query profile for bytes scanned or filter-row groups pruned, not the wall clock. If a date-filtered query scans the whole table, the problem is layout — and no amount of extra compute will fix it, it will only cost more.

6 · ENCODINGS

Encoding	How it works, and when it pays
Dictionary	Stores each distinct value once, then store small integer codes. Enormous win on low-cardinality strings — country, status, category. Writers fall back to plan encoding once the dictionary grows past a size limit, which is why it quietly stops helping on high-cardinality columns.
Run-length (RLE)	A repeated value becomes value + count. Only effective when equal values are adjacent, so its benefit is created by sort order, not by the encoder.
Bit-packing	Store integers in the minimum bits needed rather than the full type width — values 0-7 need three bits, not thirty-two. Applied to dictionary codes and small integers.
Delta	Store differences between consecutive values. Ideal for sorted timestamps and monotonically increasing values, where the deltas are tiny and highly compressible.
Frame of reference	Subtract a per-block base value, then bit-pack the remainder. Stills clustered numeric ranges.
Specialised float and string codes	Newer encodings target the two classic classic techniques handle worst: floating-point measurements and short strings. Worth checking whether your engine version supports them before assuming a column is incompressible.

Dictionary encoding, concretely

```
// country column, 1,000,000 rows, 4 distinct values
[raw: ["United States", "India", "United States", ...]]
M = ~13 bytes

dict page: ["India", "United Kingdom"]
C="United States", S="Germany"
data page: [0, 4, 2, 1, 2, 3, ...] // 2 bits each

// and if the column were SORTED, those codes
// collapse further into runs; RLE takes over
[0x2000 (0x2000) (0x2000) (0x2000)]
= four runs instead of a million values
```

Parquet chains them. An integer column is typically dictionary-encoded first, then the resulting codes are stored with a hybrid of RLE and bit-packing — runs of eight or more identical codes become a run, everything else is bit-packed. You get both wins from one column without configuring anything.

Inspect rather than assume. Parquet tooling can dump per-column encodings, compressed and uncompressed sizes per column chunk. That report tells you immediately which column is costing you the storage and whether the dictionary fell back — far more useful than reasoning about it from the schema.

Cardinality decides everything here. A UUID column defeats dictionary encoding, defaults RLE, and leaves a general-purpose code with almost nothing to work with. If a high-cardinality column dominates your storage bill, the fix is usually to stop storing it, hash it, or move it out of the hot table — not to change codecs.

7 · COMPRESSION CODECS

Encoding and compression are two different things, and listing them together is the most common conceptual error about columnar storage. Encoding is type-aware: it understands that this is a run of integers or a repeated string, and it preserves structured access. Compression is type-blind: a general-purpose codec applied to the already-encoded page bytes, looking only for repeated byte sequences.

```
// the actual write pipeline, per page
raw values
+ encode dictionary + RLE / bit-pack (type-aware)
encoded bytes
+ compresses snappy | zstd | gzip | lz4 (type-blind)
page written to disk

// on read, the reverse — but a good engine can often
// filter and aggregate on dictionary codes directly,
// without ever materialising the original strings
```

Codec	Character, and when to choose it
Snappy	Fast to compress and decompress, modest ratio. The practical default when speed matters more than storage.
ZSTD	Has largely superseded Gzip; comparable ratio at substantially faster decompression, with a tunable level. The reasonable default for cold or large data.
Gzip	Good ratio, slow to decompress. Mostly legacy now that ZSTD exists.
LZ4	Fastest, lowest ratio. Suits hot data where CPU is the constraint.
Uncompressed	Occasionally correct — when the encodings already did the work and you are CPU-bound rather than IO-bound.

The right question is not "which codec compresses best". It is where your bottleneck is. If queries are IO-bound against object storage, a heavier codec reads fewer bytes and wins. If they are CPU-bound on a warm cache, a lighter codec wins even though the files are larger.

8 · VECTORIZED EXECUTION

A classic engine processes one row at a time through a chain of operators — a virtual function call per row per operator. On a billion rows the interpretation overhead dwarfs the actual work. A vectorized engine passes batches of a column between operators instead.

```
// row-oriented: per-row dispatch, unpredictable branches
for (row : rows)
  if (filter(row)) out.add(project(row));

// vectorized: one call per batch, tight type loop
for (batch : batches)
  selection = filter(batch.col["country"]);
  sum += aggregate(batch.col["amount"], selection);
```

Why it is so much faster

- Dispatch cost is amortised across the whole batch rather than paid per row.
- The inner loop is typed and branch-free, so it vectorises to SIMD instructions that process many values per cycle.
- Cache locality is excellent — a batch of one column is contiguous memory of one type, which is exactly what a prefetcher handles well.
- Filters produce selection vectors rather than copying data, so a highly selective predicate stays cheap.
- Late materialization — carry positions and dictionary codes through the plan and fetch real values only at the end, for the rows that survived.

Batch size is a real tuning parameter. Too small and per-batch overhead ruins; too large and intermediate vectors fall out of CPU cache. Engines default to a few thousand values for that reason — sized to stay in cache, not to fill memory.

Arrow is the in-memory counterpart. Parquet is the format at rest; Arrow is the layout in memory, designed so batches can be shared between processes and languages without serialisation. That is why a Parquet-to-Arrow read path can be close to zero-copy.

10 · WHEN TO USE ONE, AND WHEN NOT TO

Strong fit

Workload	Why it fits
BI and reporting	Wide tables, few columns per query; heavy aggregation, read-mostly.
Data warehousing	Large historical fact tables loaded in batches and queried repeatedly.
Ad-hoc exploration	Unpredictable predicates over large scans, where indexing every access path is impractical.
Time series and events	Append-mostly, naturally clustered on time, filtered by time — the best possible case for skipping.
Logs and metrics	Enormous volume, low-cardinality repeated fields that compress extremely well.
Offline ML features	Large-scale training-set construction and backfills over history.
Near-real-time analytics	Workable with streaming ingest — provided compaction keeps up.

One correction worth making explicitly: a feature store is not one system. The offline side — building training sets, backfilling, point-in-time joins over history — is an ideal columnar workload. The online side serves single-entity lookups in single-digit milliseconds during inference, which is the exact access pattern columnar is worst at. Those are two stones kept consistent, not one. Listing "feature store" as a columnar use case without that split is how teams end up serving inference from a warehouse and wondering why p99 latency is unusable.

Poor fit

Workload	Why it does not fit
OLTP transactions	Single-row insert, update and delete against an immutable file layout. The impedance mismatch is fundamental, not a tuning problem.
Point lookups	Reconstructing one row means seeking into every column chunk. A row store with an index does it in one seek.
High-frequency small writes	Produces the small-file problem directly. Buffer and batch, or put a row store in front.
Serving low-latency APIs	Built for throughput on large scans, not for consistent single-digit-millisecond responses.
Highly normalised models	Many-way joins across narrow tables. Columnar engines prefer wide, denormalised tables — storage is cheap because it compresses.
SELECT + workloads	Reading every column defeats the entire advantage and adds decompression cost on top.

The usual right answer is both. An OLTP database owns the writes and the point lookups; changes flow by CDC into a columnar store that owns the analytics. Trying to serve both from one engine is the decision most often regretted — in either direction.

Volume is not the deciding factor. A ten-million-row table queried with wide scans and aggregates belongs in a columnar engine; a ten-billion-row table queried only by primary key does not. Ask what the queries look like before asking how big the data is.

12 · A QUERY, END TO END

```
SELECT country, SUM(amount)
FROM sales
WHERE event_date >= '2026-05-01'
AND event_date < '2026-06-01'
GROUP BY country;

// table: 200 columns, 5 TB, partitioned by month,
// sorted by event_date within each partition
```

#	Stage	Effect
1	Parse and plan	Two columns are required — country and amount — plus event_date for the filter. The other 197 are never considered again.
2	Partition pruning	The predicate is a bare range on the partition column, so only May's directory is listed. Eleven twelfths of the table is eliminated without opening a file.
3	File pruning	Manifest statistics drop file whose date range falls outside the window — before any footer is read.
4	Footer read	For each surviving file, the footer is fetched to get row-group offsets and statistics.
5	Row-group pruning	Min/max on event_date eliminates row groups. This step only works because the data was sorted on write.
6	Projection pushdown	Ranged GETs fetch the three columns' chunks from surviving row groups. Nothing else crosses the network.
7	Decode	Pages are decompressed and decoded into columnar batches — often still as dictionary codes rather than materialised strings.
8	Filter and aggregate	Vectorized evaluation produces a selection vector; partial aggregates are computed per task and merged.
9	Return	A handful of rows — one per country.

Every stage before the last three exists to avoid reading bytes. Of a 5 TB table, a well-laid-out version of this query might touch a few gigabytes. Change the filter to `WHERE MONTH(event_date) = 5` and steps 2, 3 and 5 all stop working — the same query then scans the whole table, on the same hardware, with the same configuration.

13 · TRAPS THAT COST THE MOST

Trap	What actually happens	Fix
Unsorted writes	Every row group's min/max spans the whole domain, so no pruning is ever possible. Statistics are written, correct, and worthless.	Sort or cluster on the leading filter column at write time; re-sort during compaction.
Small files	Each costs a footer read and a round trip; metadata grows until planning dominates; encodings have too little data to DATE(ts), CAST or UPPER hides the column, so statistics cannot be compared and pruning silently stops.	Scheduled compaction, and buffer streaming writes before flushing.
Functions on filter columns	Expects the column to be materialised.	Rewrite as a bare range against literals, or store the derived column.
SELECT *	Forfeits projection pushdown entirely — the single largest advantage — and adds decompression on every column.	Name columns. It is not style; it is most of the performance.
Over-partitioning	Partitioning on a high-cardinality column creates millions of directories and tiny files. Planning becomes slower than the scan.	Partition on one or two low-cardinality columns; cluster on the rest.
Row-by-row ingestion	An insert per event produces a file per event and a compaction backlog that never clears.	Batch on size or time; use the engine's bulk path.
Using it as OLTP	Single-row updates rewrite files or accumulate delete vectors until reads degrade badly.	OLTP database in front, CDC into the columnar store.

confirm before a columnar table carries a workload anyone depends on

11 · THE LANDSCAPE

Category

What it is: examples

File formats	What it is: examples
File formats	Columnar bytes at rest, engine-independent: Parquet, ORC . You can read them from anything; they carry no notion of a table.
In-memory format	Arrow — the columnar layout in RAM, for zero-copy sharing and SIMD execution. Not a storage format.
Table formats	Iceberg, Delta Lake, Hudi — a metadata layer over those files that provides atomic commits, snapshots, time travel and schema evolution. This is what makes a directory of files behave like a table.

Engines & warehouses

ClickHouse, Snowflake, BigQuery, Redshift, Databricks, DuckDB, Doris, StarRocks, Druid, Pinot, Vertica. These execute queries; several read the formats above directly.

They compose rather than compete. A common modern stack is Parquet files, organised by Iceberg, queried by more than one engine, with Arrow as the in-memory representation in between. Asking "Parquet or Iceberg?" or "Arrow or Parquet?" is a category error — they operate at different layers.

Rough positioning

- Real-time, high-ingest analytics — ClickHouse, Doris, StarRocks, Druid, Pinot. Built for fresh data and fast filtered aggregates.
- Cloud warehouses — Snowflake, BigQuery, Redshift. Managed, elastic, storage-compartment separated, strongest on governance and concurrency.
- Lakehouse — engines over Iceberg or Delta on object storage. Open formats, multiple engines, no vendor lock on the data itself.
- Embedded / single-node — DuckDB. Generally fast on data far larger than people expect, and it removes a cluster from many problems.

Start smaller than feels right. Single-node engines handle datasets in the hundreds of gigabytes comfortably on modern hardware. Distributed systems are chosen for concurrency, governance and elasticity at least as often as for raw size — and they cost far more to operate.

14 · DESIGN & OPERATIONS CHECKLIST

- Access pattern verified as scan-and-aggregate, not point lookup
- An OLTP store owns the writes if single-row updates are frequent
- Sort / cluster key chosen from actual query predicates
- Leading sort column is the most common filter (usually time)
- Partition columns are low cardinality, one or two levels
- Partition size distribution checked for skew before launch
- Target file size set and enforced by compaction
- Row-group size chosen deliberately, not left at a default that fights the file size
- Compaction is a scheduled, monitored job with alerting on backlog
- Streaming writes buffered on size or time, never per event
- Copy-on-write vs merge-on-read chosen on time, with the reason recorded
- Delete-file / delta accumulation monitored on merge-on-read tables

- A table format is in use if multiple writers or readers share the data
- Atomic commit verified — no reader can observe a partial write
- Snapshots retention and expiry configured; old snapshots actually expire
- Orphan file cleanup scheduled after failed writes
- Schema evolution path tested, including column rename and type widening
- Codecs chosen from the bottleneck — IO-bound or CPU-bound, measured
- High-cardinality columns identified and justified or removed
- Bloom filters enabled on high-cardinality equality predicates
- Nested fields that are filtered on promoted to top level
- Statistics refreshed after significant loads
- Query profiles reviewed for bytes scanned and row groups pruned
- No SELECT * in any scheduled or dashboard query

- No functions or casts wrapping filter columns in hot queries
- Predicate and projection pushdown confirmed in the plan, not assumed
- Compression and speed-up measured on your own data, not taken from a datasheet
- Cost model understood — bytes scanned, compute time, or storage
- Cost guardrails set so one bad ad-hoc query cannot scan the whole table
- Concurrency limits understood for the expected number of dashboard users
- Freezeless requirement stated and the ingest interval matched to it
- Backfill and reprocessing path tested at realistic volume
- Time travel / retention window agreed with whoever needs it
- Single-node engine evaluated before committing to a cluster

The one that summarises the sheet: run your slowest read query and read how many bytes it scanned against how many it needed. The gap between those two numbers is the entire subject — and it is almost always closed by changing how the data is written, not by adding compute.